



BOOTCAMP FRONTEND & REACT · SEMANA 2

Responsive

Clase 5 — Mobile first, unidades, media queries, variables CSS, modo oscuro, transiciones y animaciones

Clase virtual · L·M·V

Proyecto TechCart

STEERCL

PLAN DE HOY · EL CRITERIO ES CUBRIR EL TEMARIO, NO EL RELOJ

Agenda

1. Repaso de la Clase 4

El plano del body y la media query escrita de apuro

2. Unidades

%, em, rem, vw/vh · min, max, clamp · imagen fluida · contenedor

3. Responsive

meta viewport · modo dispositivo · @media · breakpoints

4. Mobile first

La hoja de TechCart, dada vuelta con min-width

5. Variables CSS

Tokens, alcance y el reemplazo en toda la hoja

6. Modo oscuro

prefers-color-scheme + tokens

7. Transiciones y animaciones

transition, transform, @keyframes, reduced-motion

8. Franja de beneficios

Sección nueva: integra todo

🕒 **Receso de 15 min** entre mobile first y variables · cierre: errores frecuentes, resumen, 5 tareas y anticipo

Dos de memoria

¿Qué significa `grid-column: 1 / -1`?

¿Por qué la columna del contenido es `minmax(0, 1fr)` y no `1fr` a secas?

PENDIENTE DESDE LA CLASE 3 · 1REM = 16PX · 100VH = LA VENTANA ENTERA

Las formas de medir

px fijo: mide eso, pase lo que pase

% relativo al PADRE (su contenedor)

em relativo a la letra donde estás · se ANIDA

rem relativo a la RAÍZ (html = 16px)

vh · vw % del alto / ancho de la PANTALLA

padre con letra de 12px

 6em = 72px

 6rem = 96px

padre con letra de 20px

 6em = 120px ← cambió

 6rem = 96px ← igual

em depende de dónde está parado · **rem** siempre mira la raíz. Ante la duda: **rem**.

MEDIDAS QUE CRECEN CON LA PANTALLA, PERO CON LÍMITES

min() · max() · clamp() y la imagen fluida

min(600px, 100%)

toma el **menor**: 600px, pero nunca más que el contenedor

max(1rem, 2vw)

toma el **mayor**: al menos 16px, más si la pantalla es ancha

clamp(mín, ideal, máximo)

intenta el ideal sin bajar del mínimo ni pasar del máximo

```
h2 { font-size: clamp(1.25rem, 1rem + 1.2vw, 1.5rem); }
```

20px (mínimo)

1rem + 1.2vw · fluye con el ancho

24px (máximo)

**img { max-width:100%;
height:auto }**

ninguna imagen desborda su caja; la altura sigue a la proporción

aspect-ratio · object-fit

caja con proporción fija · **contain** entera / **cover** llena y recorta

max-width + margin-inline:auto

el contenedor centrado: techo de ancho y márgenes laterales automáticos

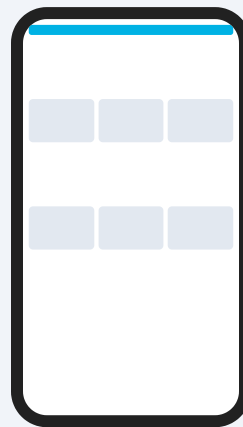
El meta viewport

```
<meta name="viewport"
      content="width=device-width,
              initial-scale=1.0">
```

width=device-width: usa tu ancho REAL como viewport.

initial-scale=1.0: arranca sin zoom.

Sin la etiqueta, el celular finge ~980px y encoge la página. Las media queries no tendrían con qué trabajar.



sin viewport
miniatura



con viewport
ancho real

UN BLOQUE DE CSS QUE SOLO EXISTE SI SE CUMPLE UNA CONDICIÓN

@media — reglas con condición

```
/* tamaño: "desde" (piso) */  
@media (min-width: 48em) { ... }  
/* tamaño: "hasta" (techo) */  
@media (max-width: 47.99em) { ... }  
/* sintaxis de rangos, moderna */  
@media (width >= 48em) { ... }  
  
/* preferencias y orientación */  
@media (prefers-color-scheme: dark) { ... }  
@media (prefers-reduced-motion: reduce) { ... }  
@media (orientation: landscape) { ... }
```

min = "desde" · **max** = "hasta".

Dentro de una media query, **em y rem valen lo mismo**: 16px del navegador.

48em = 768px · 64em = 1024px · 75em = 1200px. En em, el corte acompaña a quien agranda la letra.

Fuera de la condición, el bloque **no existe**.

MODO DISPOSITIVO (F12 → CTRL+SHIFT+M) · EL NÚMERO SALE DE MEDIR, NO DE UNA TABLA

Auditoría: de dónde sale cada corte

- 1 La tabla de tarifas pide unos **250px de ancho útil** para no partir "Internacional" → la lateral necesita **300px**
- 2 Cada tarjeta del catálogo pide **200px + 16** de separación
- 3 300 de lateral + 24 de aire + dos tarjetas + el padding pasan de 780 → **en 768 no entra; en 1024 sí**
- 4 El fieldset a dos columnas deja los campos ilegibles por debajo de unos **700px** → corte en 768

Ya era responsive sin saberlo:

- cabecera: `flex-wrap: wrap`
- catálogo: `auto-fill + minmax`
- fotos: `max-width: 100%`
- pie: `flex-wrap + order`

EL CORTE SALE DE MIRAR DÓNDE SE ROMPE EL CONTENIDO, NO DE UNA TABLA

Breakpoints **por contenido**

celular	tablet	escritorio	ancha
la base · plano de 1 columna	48em = 768px · cabecera en fila, form 2 columnas, destacada 2x2	64em = 1024px · entra la lateral de 300px	75em = 1200px · contenedor 1280px

el catálogo a mano: 1, 2, 3 columnas

```
.productos { grid-template-columns: 1fr; }

@media (min-width: 37.5em) {
  .productos { ... 1fr 1fr; }
}

@media (min-width: 48em) {
  .productos { ... repeat(3, 1fr); }
}
```

el catálogo automático (lo que tenemos)

```
.productos {
  grid-template-columns:
    repeat(auto-fill, minmax(200px, 1fr));
}
```

Va solo de 1 a 4 columnas, sin media queries.

Tarjetas iguales que se repiten → automático.

El diseño cambia de verdad (el plano) → media query.

MAX-WIDTH = TECHO ("HASTA") · MIN-WIDTH = PISO ("DESDE")

Dos direcciones, mismo resultado

desktop first: RESTAR (Clase 4)

```
/* la base es el ESCRITORIO */
body {
  grid-template-columns:
    300px minmax(0,1fr);
}

/* ...y hacia abajo se quita */
@media (max-width: 600px) {
  body {
    grid-template-columns:
      minmax(0,1fr);
  }
}
```

mobile first: SUMAR (hoy)

```
/* la base es el CELULAR */
body {
  grid-template-columns:
    minmax(0,1fr);
}

/* ...y hacia arriba se agrega */
@media (min-width: 64em) {
  body {
    grid-template-columns:
      300px minmax(0,1fr);
  }
}
```

CUSTOM PROPERTIES: DECLARO UNA VEZ, USO MIL VECES, CAMBIO EN UN SOLO LUGAR

Variables CSS: la paleta con nombre

```
/* 1. se declaran en la raíz, con dos guiones */
:root {
  --marca: #0066cc;
  --radio: 12px;
}
/* 2. se usan con var() */
h1 { color: var(--marca); }
/* 3. valor de respaldo, si no existiera */
a { color: var(--marca, #0066cc); }
/* 4. alcance: vale solo para lo de adentro */
fieldset { --radio: 8px; }
```

Cambias **una línea** (`--marca`) y cambia **toda la tienda**: título, enlaces, botones, bordes.

Design tokens: colores, espacios, radios y tipografía con nombre propio.

Se heredan por el árbol; redefinidas en un selector, mandan solo ahí adentro. JavaScript podrá leerlas y cambiarlas más adelante.

MISMOS NOMBRES, OTROS VALORES: NI UNA REGLA DE LAYOUT SE TOCA

prefers-color-scheme

```
:root { color-scheme: light dark; /*
... */ }

/* la misma paleta, de noche */
@media (prefers-color-scheme: dark)
{
  :root {
    --fondo: #111114;
    --tarjeta: #1d1d1f;
    --texto: #f5f5f7;
    --marca: #3d9bff;
    --sobre-marca: #0b1220;
    --exito: #30d158;
  }
}
```



MICRO-INTERACCIONES: CORTAS (0.1 A 0.3 S) · LA TRANSITION VA EN EL ESTADO BASE

transition — no saltes: anima

```
/* propiedad · duración · ritmo · retraso */
transition: transform .15s ease 0s;
/* varias, con coma */
transition: transform .2s ease, box-shadow
.2s ease;
/* las largas */
transition-property · -duration · -timing-
function · -delay
```

normal

Agregar

:hover

Agregar

estado A — .2s ease — estado B

SE ANIMA

color, opacity, tamaños... y sobre todo transform: translate, scale, rotate. No empuja a los vecinos.

NO SE ANIMA / NO CONVIENE

display y font-family no tienen medio camino. width, height, top se pueden, pero reacomodan todo y se traban.

Regla: la transition se escribe en el estado **base**, no en el :hover; si no, la ida es suave y la vuelta salta. Y :focus-visible para quien navega con teclado.

DE REACCIONAR (TRANSITION, 2 ESTADOS) A MOVERSE SOLO (ANIMATION, N PASOS)

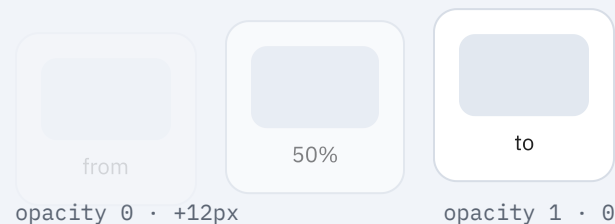
@keyframes + animation

```

.productos article {
  /* nombre · duración · ritmo · fill-mode */
  animation: aparecer .5s ease-out backwards;
}
@keyframes aparecer {
  from { opacity: 0; transform: translateY(12px); }
  to   { opacity: 1; transform: translateY(0); }
}
/* infinita y de ida y vuelta: la píldora late */
.destacado::before { animation: latido 1.6s ease-in-out infinite alternate; }

/* respeto: quien pidió menos movimiento */
@media (prefers-reduced-motion: reduce) {
  .productos article, .destacado::before {
    animation: none; }
  article, button { transition: none; }
}

```



iteration-count: 1, 3, infinite · **direction:** normal, alternate · **fill-mode:** backwards aplica el primer fotograma mientras espera su delay; forwards deja pegado el último, y por eso **both le ganaría al :hover**. will-change: solo si algo se traba de verdad.

La franja de **beneficios**



LO QUE MÁS SE ROMPE EN ESTE TEMA · NO SE CODEAN, SE RECONOCEN

Errores frecuentes

- 1 **Olvidar el meta viewport** → la página en miniatura en el teléfono y ninguna media query se activa.
- 2 **Media queries que se pisan por el orden** → en mobile first van al final y de menor a mayor; si dos se cumplen, gana la de abajo.
- 3 **Animar width, height o top** → se traba: reacomoda a todos los vecinos. Para mover, transform.
- 4 **transition: all** → anima lo que no querías (el color al cambiar de tema) y cuesta rendimiento. Nombra las propiedades.
- 5 **Variable con espacio, mayúsculas distintas o sin var()** (color: --marca) → se ignora en silencio; en DevTools sale tachada.
- 6 **Publicar movimiento sin prefers-reduced-motion** → hay personas a las que marea. La media query es obligatoria.
- 7 **Imágenes sin max-width: 100%** → desbordan su columna en el celular y aparece el scroll horizontal.
- 8 **100vh en el celular** → la barra del navegador se come un pedazo. Para eso está 100dvh.
- 9 **La transition en el :hover** → la ida es suave y la vuelta salta. Va en el estado base.

LO QUE VIMOS HOY · Y LA TIENDA QUEDÓ LISTA PARA CUALQUIER PANTALLA

Resumen

%, em, rem, vh/vw

1rem = 16px · ante la duda, rem ·
100vh y su trampa (dvh)

min() · max() · clamp()

tipografía que fluye · aire que crece ·
mínimos que no desbordan

img fluida + object-fit

max-width: 100% · aspect-ratio ·
contenedor centrado

meta viewport

el celular declara su ancho real

@media (min-width: 48em)

piso "desde" · breakpoints por
contenido · en em

mobile first

la base es el celular; las media queries
suman

:root + var()

tokens · alcance por selector · respaldo

prefers-color-scheme

modo oscuro = mismos tokens, otros
valores

transition + transform

en el estado base · sin empujar vecinos
· :focus-visible

@keyframes + animation

entrada suave y latido · fill-mode
backwards

prefers-reduced-motion

obligatoria en toda página con
movimiento

franja de beneficios

sección nueva, cero propiedades
nuevas

PARA CASA, CON LA GRABACIÓN AL LADO

5 tareas

FÁCIL

Replicar la clase + un **cuarto beneficio** con su `<article>`; comprobar a 390, 768, 1024 y 1400 que `auto-fit` lo acomoda solo.

INTERMEDIA 1

Un token nuevo, en las dos paletas: sacar la sombra del `:hover` a `--sombra` en `:root` y declararla también, más marcada, en la paleta oscura.

INTERMEDIA 2

Entrada escalonada con `animation-delay + :nth-child()` (0.1 s, 0.2 s, 0.3 s). Ahí se entiende el `backwards`. Debe apagarse con `reduced-motion`.

DIFÍCIL 1

Zoom de producto: `scale(1.05)` en la imagen con `transition`, sin salirse del marco (`overflow: hidden` en el `figure`).

DIFÍCIL 2

Cabecera pegajosa: `position: sticky + top: 0`; ojo con `z-index` y el `body` en `grid`.

Próxima clase

Clase 6 — Tailwind CSS

Cuatro clases escribiendo CSS a mano: cajas, Flexbox, Grid, responsive, tokens. Tailwind lo **comprime** en utilidades con nombre (**flex, grid, gap-4, dark:**) y es **mobile first de fábrica**: el min-width de hoy es su idioma nativo. Como ustedes ya escribieron el CSS real, van a entender qué hace cada utilidad por dentro.

Pista para casa: entra a tailwindcss.com, busca "grid-template-columns" o "dark mode" en su documentación y compara con lo que escribimos hoy.